Reusable Steps: Create ASP.NET Core Web API with MySQL Database on Mac + Rider

You can use these steps for almost every simple CRUD API project. Your PDF uses the same general flow—Model → DbContext → Connection String → Program.cs registration → Controller → Migration → Database Update → Run API—but on Mac we use terminal commands instead of Visual Studio Package Manager Console.

1. Create Web API Project

In Rider, create:

```
ASP.NET Core Web API
```

Or terminal:

```
dotnet new webapi -n MyApiProject

cd MyApiProject
```

2. Install MySQL EF Core Packages

Important: Keep EF Core and Pomelo major versions compatible.

Example using EF Core 9:

```
dotnet add package Pomelo.EntityFrameworkCore.MySql --version
9.0.0

dotnet add package Microsoft.EntityFrameworkCore --version
9.0.0

dotnet add package Microsoft.EntityFrameworkCore.Design --
version 9.0.0
```

Install EF CLI tool once on your Mac:

```
dotnet tool install --global dotnet-ef
```

Check:

```
dotnet ef --version
```

3. Create Project Folders

```
MyApiProject
│
├── Controllers
│
├── Data
│
├── Models
│
├── Program.cs
│
└── appsettings.json
```

4. Create Model

Create:

```
Models/Student.cs
```

Code:

```
namespace MyApiProject.Models;

public class Student
{
    public int Id { get; set; }

    public string StudentName { get; set; } = string.Empty;

    public string City { get; set; } = string.Empty;
}
```

The model represents your database table.

```
Student class
      ↓
Students table
```

5. Create DbContext

Create:

```
Data/AppDbContext.cs
```

Code:

```
using Microsoft.EntityFrameworkCore;
using MyApiProject.Models;

namespace MyApiProject.Data;

public class AppDbContext : DbContext
{
    public AppDbContext(
        DbContextOptions<AppDbContext> options
    ) : base(options)
    {
    }

    public DbSet<Student> Students { get; set; }
}
```

`DbSet<Student>` tells EF Core:

```
Student Model
      ↓
Students Database Table
```

6. Create MySQL Database

Open MySQL:

```
mysql -u root -p
```

Create database:

```
CREATE DATABASE API;

USE API;
```

If you are using EF Core migrations, **do not manually create the Students table**.

7. Add Connection String

Open:

```
appsettings.json
```

Add:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"server=localhost;port=3306;database=API;user=root;password=Y
OUR_PASSWORD;"
  },

  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },

  "AllowedHosts": "*"
}
```

8. Register DbContext and Controllers in Program.cs

```
using Microsoft.EntityFrameworkCore;
using MyApiProject.Data;

var builder = WebApplication.CreateBuilder(args);

var connectionString =

builder.Configuration.GetConnectionString("DefaultConnection"
);

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseMySql(
```

```
        connectionString,
        ServerVersion.AutoDetect(connectionString)
    )
);

builder.Services.AddControllers();

var app = builder.Build();

app.UseHttpsRedirection();

app.MapControllers();

app.Run();
```

Two lines are especially important:

```
builder.Services.AddControllers();
```

This **registers controllers**.

```
app.MapControllers();
```

This **activates controller routes**.

9. Check Project Builds

Run:

```
dotnet restore

dotnet build
```

Fix all errors before continuing.

10. Create Migration

```
dotnet ef migrations add InitialCreate
```

This creates:

```
Migrations/
```

The migration contains instructions for creating your database tables.

11. Apply Migration to MySQL

```
dotnet ef database update
```

Now check MySQL:

```
USE API;

SHOW TABLES;
```

You should see:

```
Students

__EFMigrationsHistory
```

12. Create Controller

Create:

```
Controllers/StudentController.cs
```

Code:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MyApiProject.Data;
using MyApiProject.Models;

namespace MyApiProject.Controllers;

[Route("api/[controller]")]
[ApiController]
public class StudentController : ControllerBase
{
    private readonly AppDbContext _context;

    public StudentController(AppDbContext context)
```

```
    {
        _context = context;
    }


    // GET ALL

    [HttpGet]
    public async Task<ActionResult<IEnumerable<Student>>>
GetStudents()
    {
        return Ok(await _context.Students.ToListAsync());
    }


    // GET BY ID

    [HttpGet("{id}")]
    public async Task<ActionResult<Student>> GetStudent(int
id)
    {
        var student = await _context.Students.FindAsync(id);

        if (student == null)
        {
            return NotFound();
        }

        return Ok(student);
    }


    // POST

    [HttpPost]
    public async Task<ActionResult<Student>>
AddStudent(Student student)
    {
        _context.Students.Add(student);

        await _context.SaveChangesAsync();

        return Ok(student);
    }
```

```
    // PUT

    [HttpPut("{id}")]
    public async Task<IActionResult> UpdateStudent(
        int id,
        Student student
    )
    {
        if (id != student.Id)
        {
            return BadRequest();
        }

        _context.Entry(student).State =
            EntityState.Modified;

        await _context.SaveChangesAsync();

        return Ok(student);
    }


    // DELETE

    [HttpDelete("{id}")]
    public async Task<IActionResult> DeleteStudent(int id)
    {
        var student =
            await _context.Students.FindAsync(id);

        if (student == null)
        {
            return NotFound();
        }

        _context.Students.Remove(student);

        await _context.SaveChangesAsync();

        return Ok(student);
    }
}
```

13. Run API

```
dotnet run
```

Terminal shows:

```
Now listening on: http://localhost:5149
```

Use the exact port shown in your terminal.

Test:

```
GET     /api/student

GET     /api/student/1

POST    /api/student

PUT     /api/student/1

DELETE  /api/student/1
```

14. Example POST JSON

```
{
  "studentName": "Meet",
  "city": "Rajkot"
}
```

15. When You Change the Model

Suppose you add:

```
public int Age { get; set; }
```

Do **not manually alter the table** when you are using migrations.

Create a new migration:

```
dotnet ef migrations add AddAgeToStudent
```

Update database:

```
dotnet ef database update
```

16. The Complete Flow to Remember

```
Create Web API Project
        ↓
Install Compatible NuGet Packages
        ↓
Create Models
        ↓
Create AppDbContext
        ↓
Create MySQL Database
        ↓
Add Connection String
        ↓
Register DbContext in Program.cs
        ↓
AddControllers()
        ↓
MapControllers()
        ↓
Build Project
        ↓
```

```
Create Migration
        ↓
Update Database
        ↓
Create Controller + CRUD APIs
        ↓
Run Project
        ↓
Test GET / POST / PUT / DELETE
```